

Simulation to hardware — written-answer draft

Review this draft against your understanding and rewrite it in your own words.

The first change is the source of truth. Our position simulator can report its command as its state; a physical arm needs measured encoder feedback with timestamps and an explicit validity/age check. The planner can remain independent of actuator details, while a hardware backend translates joint references into device commands and returns measured angles, velocities and fault information. Joint direction, zero offsets, gear ratios, units and usable limits must be calibrated and checked against the software model before running the task.

Communication and timing become control constraints. Shared actuator buses, serial transactions, delayed packets and Python scheduling can reduce the effective update rate. I would measure read/write latency and jitter, budget bus bandwidth, batch supported device transactions, and separate I/O from planning and rendering. Execution must use measured elapsed time and detect stale state. A device-side watchdog is necessary because a stalled or disconnected ROS process cannot reliably issue its own stop command. Precise low-level loops belong in suitable actuator firmware or a real-time control layer when their timing requirements demand it.

The simulated velocity actuator is only illustrative. Gravity, payload, friction, backlash, compliance and saturation can change tracking and settling. I would begin with conservative motion limits, validate actuator capabilities, tune against measured error, bound integrators and verify behavior under saturation and load. Reference acceleration limits alone do not guarantee physical acceleration or contact safety. Completion should depend on measured convergence, with explicit timeouts and fault handling, as our sequence already does.

Geometry also has uncertainty. The planner treats links as zero-width lines and assumes exact dimensions and base placement. The Gazebo backend adds finite meshes and simplified collision geometry, not a complete safety model. A physical installation needs clearance margins, finite link/gripper geometry and relevant collision checks. Ground avoidance in this planar simulation does not establish safe operation in a real workspace. Homing, controlled enable/disable and an independently effective emergency stop need defined procedures. The simulation reset must never be mapped to an instantaneous physical position change.

Finally, grasping requires evidence. Our ideal backend changes object state after a dwell; Gazebo confirms a fixed attachment and release, not frictional grasping. Real hardware needs gripper commands and appropriate confirmation of grasp/release, with recovery for missed grasps or dropped objects. The GUI should distinguish reference, measured state, stale/disconnected state, projection and actuator faults, and must not present acceptance as completion. I would progress through backend tests, unloaded low-speed motions, calibrated targets and controlled payload trials, recording tracking and timing evidence at each stage before claiming the original task transfers successfully.